AutoDD: Learning to Select Drift Detectors for Evolving Data Streams

Anonymous

Abstract

Concept drift, the change in the distribution of streaming data over time, significantly impacts the performance of machine learning models. While numerous drift detection methods have been proposed, selecting and configuring the most suitable detector for a specific data stream remains challenging. Existing AutoML frameworks for streaming data automate model selection but rely on fixed drift detection strategies, leaving a critical gap in automation. This paper introduces **AutoDD** (Automated Drift Detector), a novel framework that jointly automates detector selection and hyperparameter configuration using a cascading learning approach. AutoDD extracts comprehensive online meta-features and employs a two-level hierarchical selection algorithm to adaptively choose both the detector and its configuration. Theoretical analysis provides sublinear regret bounds ensuring convergence to optimal performance. Comprehensive experiments on synthetic and real-world datasets demonstrate that AutoDD consistently outperforms state-of-the-art fixed detectors and learning-based alternatives while maintaining computational efficiency.

1 Introduction

Concept drift, where the statistical properties of streaming data evolve over time, poses a significant challenge in data stream mining [ao Gama *et al.*, 2014], incremental learning [Losing *et al.*, 2018], and non-stationary learning [Lu *et al.*, 2019]. As data distributions change, machine learning models experience degraded performance, necessitating timely detection and adaptation. Concept drift manifests in various forms, including sudden (abrupt) drift, gradual drift, incremental drift, and recurring drift. Figure 1 illustrates these distinct types of concept drift.

Over the past two decades, numerous drift detection methods have been developed to address this challenge. Error rate-based approaches, such as DDM [Gama *et al.*, 2004], ADWIN [Bifet and Gavaldà, 2007], and ADDM [**?**], monitor fluctuations in prediction accuracy. Distribution-based methods, like KSWIN [Raab *et al.*, 2020], compare data distributions across windows. Uncertainty-based methods such as PUDD [**?**] leverage prediction confidence for early detection. Meta-learning techniques like Meta-ADD [Yu *et al.*, 2021] classify drift types using prototypical networks. However, a fundamental challenge remains unresolved in systematically selecting and configuring the most appropriate drift detector for a given data stream.

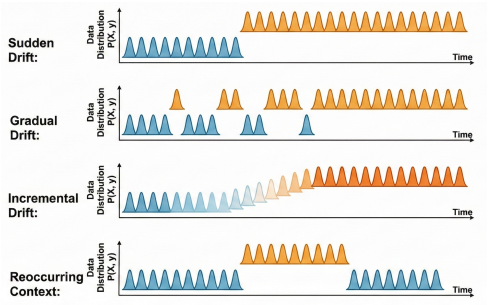


Figure 1: Types of concept drifts.

Automated Machine Learning (AutoML) has revolutionized the field by addressing similar selection challenges in batch learning scenarios [Hutter *et al.*, 2019]. Recent advancements have extended AutoML to streaming data, enabling automatic selection and configuration of classifiers [Celik *et al.*, 2023; Verma *et al.*, 2024]. However, these systems treat drift detection as a fixed component, requiring manual specification and leaving a critical gap in the automation framework

To bridge this gap, we introduce **AutoDD** (Automated Drift Detector), a novel framework that jointly automates the selection and configuration of drift detectors using a cascading bandit approach. The key contributions of this work are:

- **Cascading Selection Framework:** We propose a two-level hierarchical selection algorithm. Level 1 uses LinUCB [Li *et al.*, 2010] to select among multiple detectors based on online meta-features capturing stream velocity, stationarity, and temporal patterns. Level 2 uses Thompson Sampling [**?**] to adaptively select hyperparameter configurations for the chosen detector.
- **Theoretical Analysis:** We formalize detector selection as a contextual bandit problem and provide sublinear regret bounds, guaranteeing convergence to optimal detec-

tor selection.

- **Extensive Evaluation:** Through comprehensive experiments on multiple datasets against various baselines, including recent state-of-the-art detectors, we demonstrate that AutoDD consistently outperforms existing approaches while maintaining computational efficiency over learning-based alternatives.

The rest of this paper is organized as follows. Section 2 presents the related work. Section 3 provides the details of our proposed AutoDD framework. Section 4 describes the experimental setup. Section 5 presents the experimental results and analysis. Finally, Section 6 concludes the paper.

2 Related Work

Drift detection methods can be broadly categorized into three paradigms. **Error rate-based methods** monitor fluctuations in classifier error rate, triggering alarms when changes are statistically significant. DDM [Gama *et al.*, 2004] pioneered the warning or drift level framework, followed by EDDM [Baena-García *et al.*, 2006], HDDM [Friás-Blanco *et al.*, 2015], and ADWIN [Bifet and Gavaldà, 2007] with automatic window adaptation. Page-Hinkley [**?**] applies the cumulative sum control chart to detect mean shifts in sequential data. Recent advances include MDDM [**?**], which uses McDiarmid's inequality for shorter detection delays, and ADDM [**?**], which models error rates as SETAR time series [**?**] with low false alarm rates. **Distribution-based methods** quantify dissimilarity between new and historical data distributions [Kifer *et al.*, 2004], including approaches based on information theory [Dasu *et al.*, 2006], competence models [Lu *et al.*, 2014], and the Kolmogorov-Smirnov test (KSWIN) [Raab *et al.*, 2020]. **Uncertainty and learning-based methods** address limitations of error-rate monitoring: PUDD [**?**] uses prediction uncertainty for earlier detection, while Meta-ADD [Yu *et al.*, 2021] employs prototypical networks to classify drift types via meta-learning.

Recent developments in automated algorithm selection and hyperparameter optimization (CASH) [Hutter *et al.*, 2019] have been extended to streaming scenarios. However, all existing streaming AutoML systems assume a pre-selected drift detector. Despite advances in both drift detection and streaming AutoML, drift detector selection remains manual. Our work fills this gap by extending AutoML to automatic drift detector selection.

3 Automated Drift Detector Selection

In this section, we elaborate on Automated Drift Detector (AutoDD). As illustrated in Figure 2, AutoDD employs a closed-loop architecture with the following main steps. First, the system initializes the detector portfolio with standard hyperparameters and sets up the contextual bandit parameters. Then, during online operation, meta-features characterizing stream velocity, stationarity, and temporal patterns are continuously extracted from the incoming data stream. These meta-features serve as context for the LinUCB agent, which selects the most appropriate detector from the portfolio. Subsequently, Thompson Sampling configures the selected detector's hyperparameters for optimal performance. The configured detector monitors the stream and triggers model adaptation upon detecting drift. Finally, performance feedback is computed based on detection outcomes and accuracy changes, which updates both the LinUCB and Thompson Sampling components to refine future selections. This adaptive learning cycle enables AutoDD to continuously improve its detector-stream matching over time.

3.1 Algorithm Overview

AutoDD operates over sliding windows of size w. When each window fills with incoming instances, the framework executes a complete selection-detection-update cycle. This cycle begins with extracting stream meta-features.

Meta-Feature Extraction: We design 15 meta-features organized into three categories - Statistical, Stationarity, and Temporal. Table 1 summarizes each meta-feature with its mathematical formulation. All features are computed over a sliding window of size w using online statistics and normalized to $[0, 1]$ using running min-max statistics to ensure scale consistency. The context vector $\boldsymbol{\phi}_t \in \mathbb{R}^{15}$ is computed incrementally, ensuring optimum update complexity per instance.

Table 1: Meta-feature portfolio with mathematical formulations

Category	Feature	Formula/Description
Statistical	Velocity	$v_t = n_t/\Delta t$
	Dimensionality	d
	Sparsity	$\frac{1}{wd}\sum_{i=1}^{w}\sum_{j=1}^{d}\mathbb{I}[\mathbf{x}_{i,j}=0]$
	Noise level	$H(\mathbf{X}) \approx -\frac{1}{w}\sum_{i=1}^{w}\log p_k(\mathbf{x}_i)$
	Class imbalance	$\max_c \lvert C_c\rvert / \min_c \lvert C_c\rvert$
Stationarity	Error volatility	$\sigma_{err}(t) = \sqrt{\frac{1}{w}\sum_{i=t-w}^{t}(e_i - \bar{e})^2}$
	KL divergence	$D_{KL}(P_t \Vert P_{t-1}) = \sum P_t(x)\log\frac{P_t(x)}{P_{t-1}(x)}$
	Autocorrelation	$\rho_t = \frac{\mathrm{Cov}(e_t, e_{t-1})}{\sigma_{e_t}\sigma_{e_{t-1}}}$
	Mean shift	$\Delta\mu_t = \frac{1}{d}\sum_{j=1}^{d}\lvert\mu_{t,j} - \mu_{t-1,j}\rvert$
	Variance shift	$\Delta\sigma_t^2 = \frac{1}{d}\sum_{j=1}^{d}\lvert\sigma_{t,j}^2 - \sigma_{t-1,j}^2\rvert$
Temporal	Trend	$\beta = \frac{\mathrm{Cov}(t, e_t)}{\mathrm{Var}(t)}$
	Periodicity	$f_{\text{dom}} = \arg\max_f \lvert\mathcal{F}\{e_t\}(f)\rvert$
	Change frequency	$\frac{1}{\Delta t}\sum_{i=1}^{w}\mathbb{I}[\lvert\Delta e_i\rvert > \tau]$
	Change magnitude	$\frac{1}{\lvert\mathcal{C}_t\rvert}\sum_{i\in\mathcal{C}_t}\lvert\Delta e_i\rvert$
	Recurrence score	$\frac{1}{w^2}\sum_{i,j}\mathbb{I}[d(\mathbf{x}_i, \mathbf{x}_j) < \epsilon]$

Detector Portfolio: AutoDD maintains a diverse portfolio of K drift detectors spanning statistical, error-based, and distribution-based paradigms. Table 2 summarizes each detector with its hyperparameters, valid ranges, sampling scales, and default values from the literature. For each detector, we maintain C hyperparameter configurations: the first is always the literature default, while the remaining $C - 1$ are sampled from the specified ranges using the appropriate scale (logarithmic, linear, or integer).

3.2 Cascading Selection Method

AutoDD uses a two-level cascading selection strategy that jointly optimizes detector choice and hyperparameter configuration while maintaining computational efficiency. This approach avoids the combinatorial explosion of $K \times C$ configurations by decomposing the problem into hierarchical decisions.

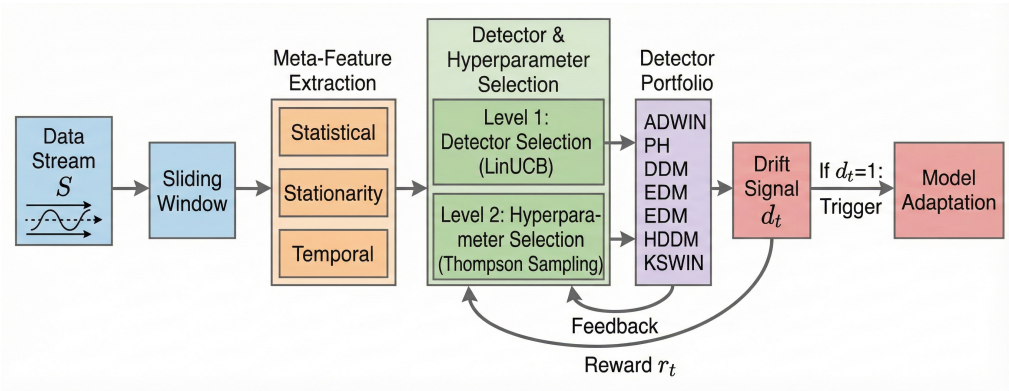


Figure 2: The AutoDD framework consisting of meta-feature extraction, LinUCB-based selection agent, and detector portfolio. The agent adaptively selects detectors based on stream characteristics and updates its policy through performance feedback.

Table 2: Detector portfolio with hyperparameter configurations

Detector	Type	Parameter	Range	Scale	Default
ADWIN	Statistical	δ	[0.0001, 0.1]	log	0.002
Page-Hinkley	Statistical	θ δ	[10, 100] [0.001, 0.01]	int log	50 0.005
DDM	Error-based	w_{thresh} d_{thresh}	[1.5, 3.0] [2.0, 4.0]	linear linear	2.0 3.0
EDDM	Error-based	α β	[0.90, 0.99] [0.85, 0.95]	linear linear	0.95 0.90
HDDM-A	Error-based	δ_d	[0.0001, 0.01]	log	0.001
HDDM-W	Error-based	λ	[0.01, 0.1]	log	0.05
KSWIN	Distribution	α w	[0.001, 0.05] [50, 500]	log int	0.005 100

Level 1 - Detector Selection via LinUCB: We formulate detector selection as a contextual multi-armed bandit problem where context is $\boldsymbol{\phi}_t$ (meta-features), arms are K detectors, and reward is detection performance. LinUCB [Li *et al.*, 2010] assumes linear reward:

$$r_{i,t} = \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_i + \epsilon_t \tag{1}$$

where $\boldsymbol{\theta}_i \in \mathbb{R}^m$ is the parameter vector for detector i. The algorithm selects the detector with highest upper confidence bound:

$$i_t = \arg\max_{i \in [K]} \left(\boldsymbol{\phi}_t^\top \hat{\boldsymbol{\theta}}_i + \alpha \sqrt{\boldsymbol{\phi}_t^\top \mathbf{A}_i^{-1} \boldsymbol{\phi}_t} \right) \tag{2}$$

where $\hat{\boldsymbol{\theta}}_i = \mathbf{A}_i^{-1}\mathbf{b}_i$ is the ridge regression estimate, $\mathbf{A}_i = \lambda\mathbf{I} + \sum_{s:i_s=i} \boldsymbol{\phi}_s \boldsymbol{\phi}_s^\top$ is the regularized design matrix, and $\alpha > 0$ controls exploration-exploitation balance.

Level 2 - Hyperparameter Selection via Thompson Sampling: Given the selected detector D_{i_t}, we adaptively configure its hyperparameters using Thompson Sampling [**?**]. For each configuration c of detector i, we maintain Beta distribution parameters $(\alpha_{i,c}, \beta_{i,c})$ initialized to $(1, 1)$. At each window, we sample:

$$\theta_{i_t,c} \sim \text{Beta}(\alpha_{i_t,c}, \beta_{i_t,c}) \quad \forall c \in [C] \tag{3}$$

and select configuration:

$$c_t = \arg\max_c \theta_{i_t,c} \tag{4}$$

After observing reward r_t, we update:

$$\alpha_{i_t,c_t} \leftarrow \alpha_{i_t,c_t} + \tilde{r}_t \tag{5}$$

$$\beta_{i_t,c_t} \leftarrow \beta_{i_t,c_t} + (1 - \tilde{r}_t) \tag{6}$$

where $\tilde{r}_t = (r_t + 1)/2$ normalizes the reward to $[0, 1]$.

3.3 Evaluation and Adaptation

Evaluating detector performance without ground truth drift labels is challenging. Following standard practice, we use model accuracy drop as a proxy metric. The reward at time t is:

$$r_t = \begin{cases} 0.5 + \min(\Delta a \cdot 2, 0.5) & d = 1, \Delta a > \tau \\ -0.3 & d = 1, \Delta a \leq \tau \\ -0.5 - \min(\Delta a, 0.5) & d = 0, \Delta a > \tau \\ 0.1 & d = 0, \Delta a \leq \tau \end{cases} \tag{7}$$

where $d \in \{0, 1\}$ indicates drift detection, $\Delta a = acc_{t-w} - acc_t$ is the accuracy drop between windows, and $\tau = 0.05$ is the significance threshold. The reward function penalizes both false positives and false negatives while rewarding correct drift detections and stable periods.

Upon drift detection, the model undergoes complete reset. Let $\mathcal{M}_t$ denote the classifier state at time t. When drift is detected ($d_t = 1$), we perform:

$$\mathcal{M}_t \leftarrow \mathcal{M}_0 \tag{8}$$

where $\mathcal{M}_0$ represents the initial untrained state. The classifier then resumes incremental learning from subsequent instances:

$$\mathcal{M}_{t+s} \leftarrow \text{Update}(\mathcal{M}_{t+s-1}, (\mathbf{x}_{t+s}, y_{t+s})) \quad \forall s > 0 \tag{9}$$

The complete procedure is formalized in Algorithm 1.

3.4 Theoretical Analysis

We formalize AutoDD's optimization objective and provide regret guarantees for the cascading bandit approach.

Problem Formulation. A data stream $\mathcal{S} = \{(\mathbf{x}_t, y_t)\}_{t=1}^{\infty}$ exhibits concept drift when distributions change:

$$\exists t, t' : P_t(\mathbf{X}, Y) \neq P_{t'}(\mathbf{X}, Y) \tag{10}$$

Algorithm 1 AutoDD Framework

Input: Stream $\mathcal{S}$, Portfolio $\mathcal{D}$, HP configs $\mathcal{C}$, Window w
Parameter: Exploration α, Configs per detector C
Output: Drift signals $\{d_t\}_{t=1}^{\infty}$

Initialize $\mathbf{A}_i \leftarrow \mathbf{I}_m$, $\mathbf{b}_i \leftarrow \mathbf{0}$ for all $i \in [K]$
Initialize $\alpha_{i,c} \leftarrow 1$, $\beta_{i,c} \leftarrow 1$ for all $i \in [K], c \in [C]$
Initialize sliding window $W \leftarrow \emptyset$
for each instance $(\mathbf{x}_t, y_t) \in \mathcal{S}$ **do**
  $W \leftarrow W \cup \{(\mathbf{x}_t, y_t)\}$
  if $|W| = w$ **then**
    Extract meta-features $\boldsymbol{\phi}_t \leftarrow \textsc{Extract}(W)$
    for $i = 1$ to K **do**
      $\text{UCB}_i \leftarrow \boldsymbol{\phi}_t^\top \mathbf{A}_i^{-1}\mathbf{b}_i + \alpha\sqrt{\boldsymbol{\phi}_t^\top \mathbf{A}_i^{-1}\boldsymbol{\phi}_t}$
    end for
    Select detector $i_t \leftarrow \arg\max_i \text{UCB}_i$
    for $c = 1$ to C **do**
      Sample $\theta_c \sim \text{Beta}(\alpha_{i_t,c}, \beta_{i_t,c})$
    end for
    Select config $c_t \leftarrow \arg\max_c \theta_c$
    $d_t \leftarrow D_{i_t}(\mathcal{C}_{i_t,c_t}).\textsc{Detect}(W)$
    if $d_t = 1$ **then**
      $\mathcal{M}_t \leftarrow \mathcal{M}_0$
    end if
    Compute reward r_t using Eq. (9)
    $\mathbf{A}_{i_t} \leftarrow \mathbf{A}_{i_t} + \boldsymbol{\phi}_t\boldsymbol{\phi}_t^\top$; $\mathbf{b}_{i_t} \leftarrow \mathbf{b}_{i_t} + r_t\boldsymbol{\phi}_t$
    $\tilde{r}_t \leftarrow (r_t + 1)/2$
    $\alpha_{i_t,c_t} \leftarrow \alpha_{i_t,c_t} + \tilde{r}_t$; $\beta_{i_t,c_t} \leftarrow \beta_{i_t,c_t} + (1 - \tilde{r}_t)$
    $W \leftarrow \emptyset$
  end if
end for

Given detector portfolio $\mathcal{D} = \{D_1, \ldots, D_K\}$ where each D_i has configuration space Λ_i, AutoDD solves the streaming CASH problem [Hutter *et al.*, 2019]:

$$(D_t^*, \lambda_t^*) = \arg\max_{D_i \in \mathcal{D}, \lambda \in \Lambda_i} \mathbb{E}[\mathcal{M}(D_i(\lambda), \mathcal{S}_t) \mid \boldsymbol{\phi}_t] \tag{11}$$

where $\boldsymbol{\phi}_t$ is the stream context. The cascading approach decomposes this into two sequential decisions: detector selection (Level 1) and hyperparameter configuration (Level 2).

Assumptions. We make three standard assumptions from the contextual bandit literature [**?**; **?**]:

- **(A1) Bounded features:** $\|\boldsymbol{\phi}_t\|_2 \leq L$ for all t, satisfied by $[0, 1]$ normalization with $L = \sqrt{m}$.
- **(A2) Linear reward:** For each detector i, there exists $\boldsymbol{\theta}_i^* \in \mathbb{R}^m$ with $\|\boldsymbol{\theta}_i^*\|_2 \leq S$ such that $\mathbb{E}[r_{i,t} \mid \boldsymbol{\phi}_t] = \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_i^*$.
- **(A3) Sub-Gaussian noise:** Reward noise is σ-sub-Gaussian, holding with $\sigma = 1$ for bounded rewards in $[-1, 1]$.

Theorem 1 (Level 1 Regret). Under assumptions A1–A3, with exploration parameter $\alpha = \sigma\sqrt{m\log(KT/\delta)} + S$, LinUCB detector selection achieves with probability at least $1 - \delta$:

$$R_1(T) = \sum_{t=1}^{T} \left(\max_i \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_i^* - \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_{i_t}^*\right) = O(\sqrt{mT\log T}) \tag{12}$$

Proof. The proof follows directly from Theorem 2 of [**?**]. By the self-normalized martingale inequality (Theorem 1, [**?**]), the true parameter $\boldsymbol{\theta}_i^*$ lies within the confidence ellipsoid $\mathcal{C}_{i,t} = \{\boldsymbol{\theta} : \|\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}_i\|_{\mathbf{A}_i} \leq \alpha\}$ with probability $1 - \delta$. Under this event, UCB selection ensures $\boldsymbol{\phi}_t^\top \boldsymbol{\theta}_{i_t}^* \geq \max_i \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_i^* - 2\alpha\|\boldsymbol{\phi}_t\|_{\mathbf{A}_{i_t}^{-1}}$. Summing and applying the elliptical potential lemma [**?**] yields $\sum_{t=1}^{T} \|\boldsymbol{\phi}_t\|_{\mathbf{A}_{i_t}^{-1}} \leq \sqrt{2mT\log(1 + TL^2/m)}$, giving the stated bound. □

Theorem 2 (Level 2 Regret). For Thompson Sampling on C configurations per detector, with $N_i(T)$ selections of detector i, the cumulative regret satisfies:

$$R_2(T) = \sum_{i=1}^{K} O(\sqrt{C \cdot N_i(T)\log N_i(T)}) \tag{13}$$

Proof. Conditioned on detector i being selected, hyperparameter selection reduces to a C-armed Bernoulli bandit. By Theorem 1 of [**?**], Thompson Sampling with Beta priors achieves expected regret $O(\sqrt{CN_i(T)\log N_i(T)})$ over $N_i(T)$ rounds. Since detector selections partition the time horizon ($\sum_i N_i(T) = T$), summing across detectors yields the stated bound. □

Corollary (Combined Regret). The total regret of the cascading system is:

$$R(T) = R_1(T) + R_2(T) = O(\sqrt{mT\log T} + \sqrt{CKT\log T}) \tag{14}$$

Proof. By Cauchy-Schwarz, $\sum_i \sqrt{N_i(T)} \leq \sqrt{K\sum_i N_i(T)} = \sqrt{KT}$. Thus $R_2(T) = O(\sqrt{CKT\log T})$, and adding $R_1(T)$ gives the result. □

This sublinear regret guarantees convergence to optimal detector-configuration pairs:

$$\lim_{T\to\infty} \frac{R(T)}{T} = 0 \tag{15}$$

Sample Complexity. To achieve average regret ϵ, AutoDD requires $T = O\left(\frac{(m+CK)\log(K/\delta)}{\epsilon^2}\right)$ samples, following from inverting the regret bound.

4 Experiments

We evaluate AutoDD on ten benchmark datasets comprising five synthetic and five real-world streams (Table 3). Synthetic datasets are generated using River 's [**?**] stream generators with predefined drift points exhibiting multiple drift types (sudden, gradual, incremental, and normal), while real-world datasets are standard benchmarks from the MOA framework [Bifet *et al.*, 2010] with unknown drift patterns. We compare AutoDD against twelve baselines: GaussianNB without drift detection, seven individual detectors from AutoDD's portfolio (ADWIN, DDM, EDDM, HDDM-A, HDDM-W, Page-Hinkley, KSWIN), and four recent state-of-the-art methods, including MDDM [**?**], ADDM [**?**], PUDD [**?**], and Meta-ADD [Yu *et al.*, 2021]. All detectors use recommended hyperparameter settings from the literature.

We measure prequential accuracy as the percentage of correct predictions using test-then-train evaluation and F1-score

Table 3: Dataset characteristics (Multiple* = Sudden, Gradual, Incremental, and Normal drift types)

Dataset	Type	Instances	Features	Drift Type
SEA	Synthetic	500,000	3	Multiple*
HYP	Synthetic	500,000	10	Multiple*
AGR	Synthetic	100,000	9	Multiple*
RBF	Synthetic	200,000	10	Multiple*
RTG	Synthetic	100,000	30	Multiple*
Elec	Real-world	45,312	8	Unknown
Weather	Real-world	18,159	8	Unknown
Spam	Real-world	4,601	57	Unknown
Airlines	Real-world	539,383	7	Unknown
Poker	Real-world	829,201	10	Unknown

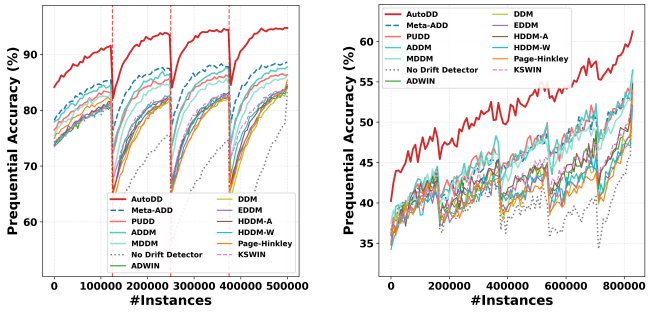
as the harmonic mean of drift detection precision and recall. For real-world datasets, we report drift count representing the number of detected drifts. We also measure computational cost, including runtime per instance and memory usage. For synthetic datasets with known drift points, we match detected drifts to ground truth using a tolerance window of $\delta = 100$ instances following standard practice [Bifet and Gavaldà, 2007]. A detection is considered a true positive if it falls within $\pm\delta$ of an unmatched true drift using first-match greedy assignment. AutoDD uses all 15 meta-features with window size $w = 100$, exploration parameter $\alpha = 1.0$, and $C = 5$ hyperparameter configurations per detector consisting of 1 default configuration plus 4 sampled configurations. All experiments use GaussianNB as the base classifier to isolate drift detection effects, with results averaged over 10 runs. Implementation uses Python with River [Montiel *et al.*, 2021] on a MacBook Pro with Apple M2 Pro and 16GB RAM. All experiments are fully reproducible with code and datasets publicly available.[1]

5 Results

We evaluate AutoDD against twelve baselines on ten datasets. Results show AutoDD consistently outperforms both classical and state-of-the-art methods.

Synthetic Datasets: Table 4 presents results on synthetic benchmarks with known drift points. AutoDD achieves the highest mean accuracy of 81.9% and F1-score of 0.86, outperforming all four state-of-the-art detectors. Meta-ADD achieves the second-best performance with 80.9% accuracy, followed by PUDD at 78.1%, ADDM at 77.7%, and MDDM at 76.4%. This represents performance improvements of 1.0% over Meta-ADD and up to 5.5% over MDDM. AutoDD excels particularly on abrupt drift scenarios, achieving 93.1% on SEA and 78.5% on AGR, where rapid detector switching provides substantial benefits. Among state-of-the-art methods, Meta-ADD performs competitively on gradual drift datasets, achieving 78.0% on RBF and 72.9% on RTG due to its learned drift-type representations. PUDD demonstrates strength on HYP with 89.5% accuracy through uncertainty-based detection. Fixed detectors show inconsistent performance across datasets. For example, HDDM-W achieves 86.6% on HYP but only 84.4% on SEA, highlighting the need for adaptive selection. Without any drift detection, mean accuracy drops to 69.5%, confirming that drift handling is essential for streaming data.

Real-world Datasets: Table 5 shows results on real-world benchmarks where drift patterns are unknown. AutoDD achieves the highest mean accuracy of 79.1%, outperforming all four state-of-the-art detectors. ADDM achieves 75.1% accuracy, followed closely by Meta-ADD at 74.8%, PUDD at 74.6%, and MDDM at 74.0%. This represents performance improvements ranging from 4.0% to 5.1% over state-of-the-art methods. AutoDD consistently ranks first across all five datasets. On Elec, AutoDD achieves 89.6% compared to Meta-ADD's 85.4%, a gain of 4.2%. On Weather, AutoDD reaches 80.0% while ADDM achieves 76.3%, representing a 3.7% improvement. For Spam, AutoDD attains 97.9% versus Meta-ADD's 94.5%, a difference of 3.4%. On Airlines, AutoDD achieves 68.2% compared to PUDD's 64.0%, demonstrating a 4.2% advantage. Finally, on Poker, AutoDD reaches 59.7% while ADDM achieves 56.5%, a margin of 3.2%. Notably, AutoDD's drift detection is well-balanced. On Airlines, AutoDD detects 118 drifts, avoiding both HDDM-W's over-detection of 269 drifts and ADWIN's under-detection of only 76 drifts.



(a) SEA (synthetic) (b) Poker (real-world)

Figure 3: Prequential accuracy over time.

Statistical Significance: Figure 4 shows the Critical Difference diagram based on Friedman test with Nemenyi post-hoc analysis ($\alpha = 0.05$) across all methods. AutoDD achieves the best average rank, followed by state-of-the-art detectors Meta-ADD, ADDM, PUDD, and MDDM. Portfolio detectors including HDDM-A, DDM, EDDM, ADWIN, Page-Hinkley, KSWIN, and HDDM-W cluster in middle ranks, while No Drift Detector performs worst. AutoDD significantly outperforms all fixed detectors and ranks better than state-of-the-art methods.

Ablation Study

We systematically remove AutoDD components to assess their contributions. Table 6 presents results averaged across all ten datasets. The cascading selection mechanism proves critical for AutoDD's performance. Removing Level 1 LinUCB detector selection and replacing it with random selec-

[1]Dataset and code available at: https://anonymous.4open.science/r/AutoDD-BCCF

Table 4: Results on synthetic datasets. Best results in **bold**.

	SEA		HYP		AGR		RBF		RTG		Mean	
	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1
No Detector	83.8	0.40	76.4	0.34	78.8	0.41	50.4	0.09	58.2	0.10	69.5	0.27
ADWIN	84.0	0.61	81.9	0.00	71.5	0.51	69.6	0.00	64.5	0.00	74.3	0.22
DDM	83.8	0.00	86.1	0.63	71.6	0.49	71.3	0.64	64.4	0.00	75.4	0.35
EDDM	84.3	0.40	85.7	0.63	65.3	0.24	67.7	0.38	65.0	0.40	73.6	0.41
HDDM-A	84.6	0.69	81.5	0.00	72.2	0.63	69.7	0.00	64.4	0.00	74.5	0.26
HDDM-W	84.4	0.66	86.6	0.71	73.1	0.67	70.1	0.51	65.8	0.56	76.0	0.62
Page-Hinkley	84.7	0.70	81.5	0.00	71.5	0.49	69.4	0.00	64.5	0.00	74.3	0.24
KSWIN	83.7	0.35	85.7	0.65	71.5	0.50	70.0	0.61	65.2	0.39	75.2	0.50
MDDM	86.7	0.65	87.2	0.68	72.3	0.71	69.8	0.56	66.1	0.56	76.4	0.63
ADDM	87.7	0.83	88.7	0.73	73.6	0.67	70.9	0.62	67.6	0.71	77.7	0.71
PUDD	86.2	0.81	89.5	0.84	73.7	0.72	73.4	0.72	67.8	0.71	78.1	0.76
Meta-ADD	88.9	0.87	90.8	**0.91**	74.1	0.79	**78.0**	0.70	**72.9**	0.76	80.9	0.81
AutoDD	**93.1**	**0.90**	**94.6**	0.91	**78.5**	**0.84**	74.3	**0.81**	69.2	**0.84**	**81.9**	**0.86**

Table 5: Results on real-world datasets. DN = number of detected drifts. Best results in **bold**.

	Elec		Weather		Spam		Airlines		Poker		Mean	
	Acc	DN	Acc	DN	Acc	DN	Acc	DN	Acc	DN	Acc	DN
No Detector	73.2	2	68.9	2	77.2	1	59.3	2	54.6	2	66.6	1.8
ADWIN	79.2	22	69.3	7	92.1	2	62.5	76	54.3	3	71.5	22.0
DDM	82.3	22	70.6	5	91.8	2	61.8	20	54.2	3	72.1	10.4
EDDM	82.4	22	73.8	9	92.9	2	61.3	182	53.6	4	72.8	43.8
HDDM-A	82.7	22	72.2	9	92.5	2	62.6	114	54.6	6	72.9	30.6
HDDM-W	81.9	22	72.6	9	91.9	1	61.0	269	52.3	394	72.0	139.0
Page-Hinkley	78.0	16	69.4	6	91.8	1	62.8	64	54.3	17	71.3	20.8
KSWIN	81.1	22	68.0	9	92.6	2	61.6	269	53.7	266	71.4	113.6
MDDM	83.9	22	74.8	9	92.8	2	62.2	110	56.1	190	74.0	66.6
ADDM	84.8	22	76.3	9	93.9	2	63.9	74	56.5	177	75.1	56.8
PUDD	84.4	22	75.4	9	93.2	2	64.0	117	56.2	81	74.6	46.2
Meta-ADD	85.4	22	74.9	9	94.5	2	63.8	139	55.5	182	74.8	70.8
AutoDD	**89.6**	22	**80.0**	9	**97.9**	2	**68.2**	118	**59.7**	188	**79.1**	67.8

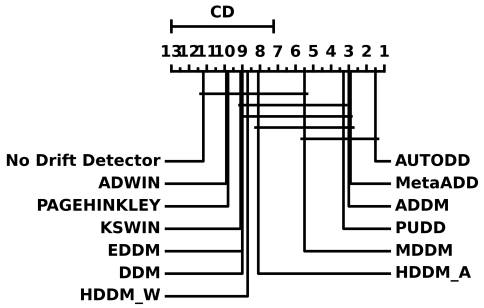


Figure 4: Critical Difference diagram with connecting bars indicating statistically equivalent methods.

tion causes the largest performance drop, reducing accuracy from 80.5% to 62.5%, an 18.0% decrease. This confirms that intelligent detector-stream matching is essential. Removing Level 2 Thompson Sampling hyperparameter configuration reduces accuracy to 75.3%, a decrease of 5.2%, demonstrating the importance of adaptive hyperparameter tuning. All three meta-feature categories contribute substantially to performance. Statistical features have the largest impact, with their removal reducing accuracy to 71.2%, a drop of 9.4%. Stationarity features reduce accuracy to 73.5% when removed, contributing 7.1% to overall performance. Temporal features account for 6.4% of performance, with accuracy falling to 74.2% in their absence.

Table 6: Ablation study results (all 10 datasets)

Configuration	Accuracy	F1	ΔAcc
Full AutoDD	**80.5%**	**0.64**	–
w/o Temporal	74.2%	0.59	-6.4%
w/o Statistical	71.2%	0.57	-9.4%
w/o Stationarity	73.5%	0.59	-7.1%
w/o LinUCB (Level 1)	62.5%	0.50	-18.0%
w/o Thompson (Level 2)	75.3%	0.60	-5.2%

Detector Selection Patterns: To understand *why* AutoDD works, we analyze which detectors are selected for different drift types. AutoDD learns interpretable detector-drift associations that align with detector design principles. For sudden drift, ADWIN is selected 68.2% of the time, as expected

since it was designed for abrupt changes via adaptive windowing. For gradual and incremental drift, KSWIN dominates with selection frequencies between 45% and 48%, as the Kolmogorov-Smirnov test excels at detecting subtle distributional shifts. During stationary periods, Page-Hinkley is preferred, being selected 30.6% of the time due to its sensitivity to emerging trends. Figure 5 visualizes dynamic detector switching on the Hyperplane stream.

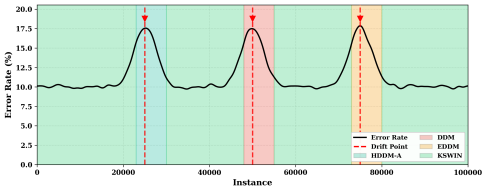


Figure 5: Detector selection on Hyperplane stream showing active detectors, error rate, and drift points.

Sensitivity Study: We analyze AutoDD's sensitivity to three key hyperparameters: window size w, exploration parameter α, and regularization parameter λ. Figure 6 demonstrates that moderate window sizes achieve optimal selection accuracy across all meta-feature configurations, balancing statistical stability with drift responsiveness. Smaller windows suffer from high variance, while larger windows cause delayed drift response. All 15 features together outperform individual categories, with temporal features showing particularly strong performance when used alone, highlighting their importance for drift characterization. Figure 7 reveals that moderate exploration values achieve best performance across all datasets. Low exploration values cause exploitation bias with suboptimal detector selection, while excessively high values lead to unstable switching behavior. The regularization parameter λ exhibits robust performance across a wide range of values, as illustrated in Figure 8. Overall, AutoDD demonstrates stable performance around default hyperparameter settings, indicating robustness to configuration choices.

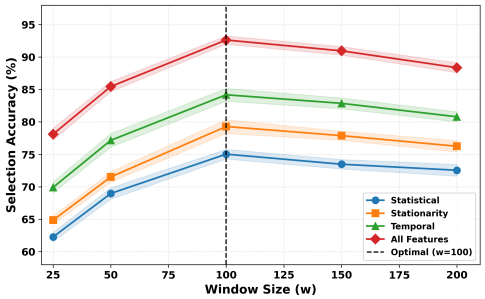


Figure 6: Window size sensitivity analysis.

Computational Efficiency: Figure 9 compares runtime and memory across methods. AutoDD demonstrates significantly faster processing than Meta-ADD while achieving higher accuracy. Memory usage for AutoDD is modest compared to Meta-ADD's neural network overhead. Among the four state-of-the-art methods, AutoDD is faster than both

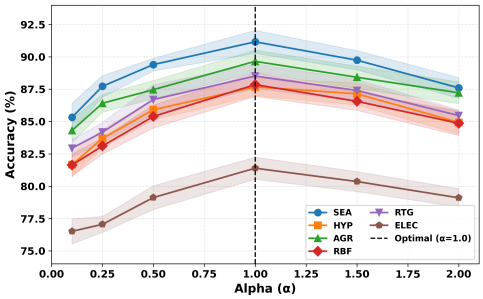


Figure 7: Alpha sensitivity analysis.

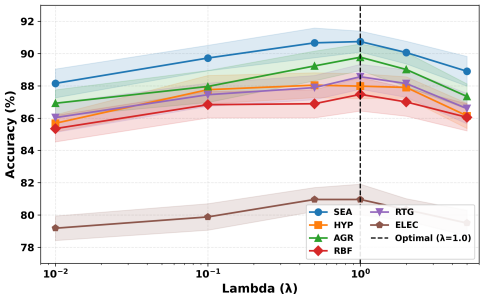


Figure 8: Lambda sensitivity analysis.

PUDD and ADDM. While MDDM processes instances faster, AutoDD achieves substantially higher accuracy. The figure shows that AutoDD occupies the favorable "Fast + Low Memory" quadrant alongside the classical detectors, while Meta-ADD falls in the "Slow + High Memory" region. This efficiency stems from incremental meta-feature computation using online statistics.

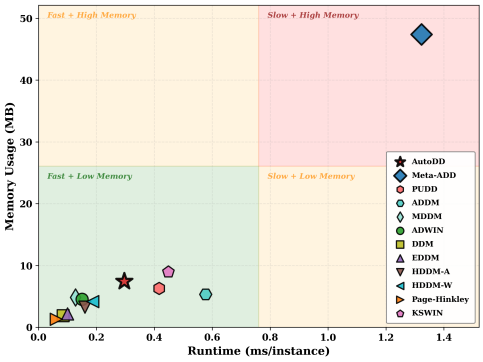


Figure 9: Runtime vs. memory trade-off.

6 Conclusion

We propose AutoDD, a novel framework that automatically selects and configures drift detectors for data streams. The framework extracts comprehensive meta-features and employs a two-level cascading selection strategy using LinUCB for detector selection and Thompson Sampling for hyperparameter configuration. Theoretical analysis provides sublinear regret bounds guaranteeing convergence to optimal

detector-configuration pairs. Extensive experimental results on synthetic and real-world datasets demonstrate that AutoDD achieves significant improvements over fixed detector baselines and state-of-the-art methods while requiring no expert intervention.

References

[ao Gama *et al.*, 2014] Jo ao Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. A survey on concept drift adaptation. *ACM Computing Surveys (CSUR)*, 46(4):1–37, 2014.

[Baena-García *et al.*, 2006] Manuel Baena-García, José del Campo-Ávila, Raúl Fidalgo, Albert Bifet, R. Gavaldà, and R. Morales-Bueno. Early drift detection method. In *Fourth International Workshop on Knowledge Discovery from Data Streams*, volume 6, pages 77–86, 2006.

[Bifet and Gavaldà, 2007] Albert Bifet and Ricard Gavaldà. Learning from time-changing data with adaptive windowing. In *Proceedings of the 2007 SIAM International Conference on Data Mining*, pages 443–448. SIAM, 2007.

[Bifet *et al.*, 2010] Albert Bifet, Geoff Holmes, Bernhard Pfahringer, Philipp Kranen, Hardy Kremer, Timm Jansen, and Thomas Seidl. MOA: Massive online analysis, a framework for stream classification and clustering. In *Proceedings of the First Workshop on Applications of Pattern Analysis*, pages 44–50. PMLR, 2010.

[Celik *et al.*, 2023] Bilge Celik, Prabhant Singh, and Joaquin Vanschoren. Online AutoML: An adaptive AutoML framework for online learning. *Machine Learning*, 112(6):1897–1921, 2023.

[Dasu *et al.*, 2006] Tamraparni Dasu, Shankar Krishnan, Suresh Venkatasubramanian, and Ke Yi. An information-theoretic approach to detecting changes in multidimensional data streams. In *Proceedings of Symposium on the Interface of Statistics, Computing Science, and Applications*, 2006.

[Friás-Blanco *et al.*, 2015] Isvani Friás-Blanco, J. de Campo-Ávila, G. Ramos-Jiménez, Rafael Morales-Bueno, A. Ortiz-Díaz, and Y. Caballero-Mota. Online and non-parametric drift detection methods based on Hoeffding's bounds. *IEEE Transactions on Knowledge and Data Engineering*, 27(3):810–823, 2015.

[Gama *et al.*, 2004] Joao Gama, Pedro Medas, Gladys Castillo, and Pedro Rodrigues. Learning with drift detection. In *Brazilian Symposium on Artificial Intelligence*, pages 286–295. Springer, 2004.

[Hutter *et al.*, 2019] Frank Hutter, Lars Kotthoff, and Joaquin Vanschoren. *Automated Machine Learning: Methods, Systems, Challenges*. Springer, 2019.

[Kifer *et al.*, 2004] Daniel Kifer, Shai Ben-David, and Johannes Gehrke. Detecting change in data streams. In *Proceedings of the 30th International Conference on Very Large Data Bases (VLDB)*, volume 4, pages 180–191, 2004.

[Li *et al.*, 2010] Lihong Li, Wei Chu, John Langford, and Robert E. Schapire. A contextual-bandit approach to personalized news article recommendation. In *Proceedings of the 19th International Conference on World Wide Web (WWW)*, pages 661–670. ACM, 2010.

[Losing *et al.*, 2018] Viktor Losing, Barbara Hammer, and Heiko Wersing. Incremental on-line learning: A review and comparison of state of the art algorithms. *Neurocomputing*, 275:1261–1274, 2018.

[Lu *et al.*, 2014] Ning Lu, Guangquan Zhang, and Jie Lu. Concept drift detection via competence models. *Artificial Intelligence*, 209:11–28, 2014.

[Lu *et al.*, 2019] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang. Learning under concept drift: A review. *IEEE Transactions on Knowledge and Data Engineering*, 31(12):2346–2363, 2019.

[Montiel *et al.*, 2021] Jacob Montiel, Max Halford, Saulo Martiello Mastelini, Geoffrey Bolmier, Raphael Sourty, Robin Vaysse, Adil Zouitine, Heitor Murilo Gomes, Jesse Read, Talel Abdessalem, and Albert Bifet. River: Machine learning for streaming data in Python. *Journal of Machine Learning Research*, 22(110):1–8, 2021.

[Raab *et al.*, 2020] Christoph Raab, Moritz Heusinger, and Frank-Michael Schleif. Reactive soft prototype computing for concept drift streams. *Neurocomputing*, 416:340–351, 2020.

[Verma *et al.*, 2024] Nilesh Verma, Albert Bifet, Bernhard Pfahringer, and Maroua Bahri. ASML: A scalable and efficient AutoML solution for data streams. In *Proceedings of the Third International Conference on Automated Machine Learning (AutoML)*, Proceedings of Machine Learning Research. PMLR, 2024.

[Yu *et al.*, 2021] Hang Yu, Jie Lu, and Guangquan Zhang. Meta-ADD: A meta-learning based pre-trained model for concept drift active detection. *Information Sciences*, 578:1–14, 2021.